Oioi, esse arquivo faz parte do desenvolvimento do PSET 1. Eu fiz um roteiro antes de passar pra folha almaço que o Abrantes pediu (e recomendo que você faça o mesmo, poupa bastante tempo).

Como ele não deu a folha de volta, vou disponibilizar esse arquivo. Ah, mas cuidado que eu mudei algumas poucas coisas antes de passar pra folha.

Questão 1: O que são sistemas de controles de versões? Por que são importantes?

Os sistemas de controles de versões são softwares que acompanham com si ferramentas as quais monitoram e rastreiam alterações em um código fonte. Estes auxiliam equipes e desenvolvedores dando agilidade e flexibilidade em suas implementações. Entre seus mecanismos está a preservação das alterações feitas em um código, formando um histórico, bem como a visualização de mudanças, além disso, dá a possibilidade que outros usuários contribuam, experimentem e comentem sobre o código.

Questão 2: Qual a diferença entre o Git e o GitHub? Como eles estão relacionados? É possível usar um sem o outro?

A diferença principal entre o Git e o GitHub são suas funções. O Git é um software de sistema de controle de versão (como elucidado na questão 1), enquanto o GitHub utiliza as funções e o software Git para compor a sua plataforma de gerenciamento de código, servindo de repositório remoto para desenvolvedores. Pelo fato do GitHub utilizar a tecnologia Git, não seria possível a permanência de um sem o subsídio do outro.

Questão 3: O Git é um sistema distribuído de controle de versões. O que significa isso?

Ser um sistema de controle de versões distribuído significa que todo o projeto é espelhado para todos os outros usuários conectados ao servidor Git. Entre as vantagens comparando-se ao modelo centralizado estão a velocidade e o acesso offline. A maioria das aplicações open-source utilizam-se do sistema distribuído.

Questão 4: Existem alguns errinhos no projeto HR ilustrado acima(principalmente em relação às cardinalidades dos relacionamentos e em relação à obrigatoriedade das chaves estrangeiras). Identifique quais são esses erros, explique o que está errado e conserte os erros se for necessário.

Erro 1: Na tabla países, o ID (que funciona como chave primária), na imagem fornecida aparece com precisão 2, enquanto no arquivo hr.html ela não tem. Vejo como mais viável não ter precisão, afinal, um ID pode sim ter mais de dois caracteres

Erro 2: Ao associar uma primary key a outra tabela, a foreign key deve conter o argumento “NOT NULL”. Nos arquivos fornecidos não há isso

Erro 3: Na tabela localizações, o campo uf não poderia ter a redundancia de adicionar a sigla ou o nome. Para facilitar, adicionei apenas a sigla

Erro 4: Existe uma ligação da tabela empregados e departamentos que não devia existir. Empregados deve ter a chave estrangeira de departamentos, mas em departamentos não precisa ter um de empregado

Erro 5: Na descrição da tabela departamentos, o campo id_localizacao e id_gerente possuem suas descrições trocadas

redundancia

Erro 6: CEP tem 6 números

Questão 5: Alguma tabela do projeto representa um relacionamento do tipo N:N? Se sim, identifique a tabela e explique porque é um relacionamento N:N; Se não, explique porque não há relacionamentos N:N neste projeto.

Sim, existem relacionamentos do tipo N:N no projeto, o relacionamento entre departamentos, cargos e empregos.

- Um departamento pode ter vários cargos. E um cargo pode ter vários departamentos
- Um departamento pode ter vários empregados. E um empregado pode estar em vários departamentos.
- Um empregado pode ter vários cargos. E um cargo pode estar em vários empregados.

Tudo isso é possível devido a uma “quarta entidade”, a entidade associativa, que associa outras duas entidades por meio de dois relacionamentos 1:N. Essa entidade é a histórico cargos

Questão 6: Pelo projeto apresentado é possível que um determinado empregado seja o gerente de zero, um ou mais departamentos. Se eu quisesse impor uma regra que diz que um empregado só pode ser gerente de, no máximo, um departamento, o que eu teria que fazer no projeto? O que eu teria que criar no banco de dados para impor essa restrição? Por quê?

Para impor essa regra a cardinalidade do relacionamento entre as chaves “id_gerente” e “id_empregado” das tabelas departamentos e empregados deve mudar para 1:1, onde um departamento pode ter somente um “id_gerente” igual a um outro “id_empregado”

Questão 7: Por que os relacionamentos entre as tabelas estão representados com linhas pontilhadas? O que isso representa? Por que foi representado assim?

(mudar escrita depois, foi Google Translate)

As linhas pontilhadas revelam que a relação é não identificada, isso significa que os atributos de chave primária do pai não devem se tornar atributos de chave primária do filho. Os relacionamentos não identificados podem ser classificados como “mandatory” (não achei uma tradução válida para o português) ou opcionais. Um relacionamento “mandatory” é válido quando o valor na tabela filho não pode ser nulo.

Na outra mão, um relacionamento identificado ocorre quando a existência de uma coluna em uma tabela filha depende de uma coluna em uma tabela pai. Essa representação no SQL Power Architect é feita por uma ligação contínua.

Questão 8: Qual é o único tipo de relacionamento que pode guardar dados? Por quê? Existe algum relacionamento assim nesse projeto? Se não existir, você sugeriria trocar alguns dos relacionamentos existentes para melhorar o projeto?

Questão 9: Explique o relacionamento da tabela empregados com ela mesma? É um erro? É correto? Por quê?

Na tabela empregados, a coluna id_supervisor se relaciona com a própria tabela e a coluna id_empregado.

E não, não é um erro, de certa forma o supervisor também pertence ao grupo de empregados.

Ademais, auto relacionamentos representam de certa forma relações de hierarquia entre ocorrências de uma mesma entidade.

Questão 10: Qual a diferença entre banco de dados, usuário e esquema no PostgreSQL?

Um esquema é um grupo de tabelas relacionadas e um banco de dados é uma coleção de esquemas. Um esquema é uma estrutura lógica para armazenar dados em um banco de dados.

O termo “esquema de banco de dados” refere-se a uma representação visual de um banco de dados, um conjunto de regras que governa um banco de dados ou todo o conjunto de objetos pertencentes a um determinado usuário.

Questão 11: Por que um esquema é importante?

Questão 12: Se você não definir um esquema específico, onde os objetos do banco de dados (tabelas, relacionamentos, dados, etc.) serão gravados? Isso é bom ou ruim? Por que?

Claramente será uma decisão ruim e precipitada, afinal, grandes são as chances de serem armazenados dados e tabelas que não possuem relação entre si, removendo-se a premissa básica de um banco de dados que é possuir dados que contenham uma conexão.

Questão 13: Agora que você já implementou o projeto no PostgreSQL, tem alguma

sugestão de melhoria a fazer para o projeto? Como ele poderia ser melhorado?

Uma das melhorias seria adicionar um “salário médio” no campo cargos, fazendo uma média dos salários dependendo de cada cargo e da quantidade de funcionários que fazem parte deste cargo.

Questão 14: Faça uma comparação dos SGBD que você utilizou. Quais as vantagens e desvantagens de cada um? Quem tem a melhor documentação?

Entre o MySQL/MariaDB e o Postgres, achar informações e tutoriais do Postgres, nas minhas pesquisas pelo menos, foi bem mais simples, com uma vasta comunidade no Stackoverflow e outros fóruns. Procurando sobre documentação pareceu que o MySQL tinha muito mais, mas, como disse, nas minhas procuras o Postgres foi bem mais fácil de encontrar tutoriais e pessoas sanando dúvidas na internet.

Questão 15: Como você acha que foi seu desempenho neste PSet? Como você foi buscando as informações necessárias e lendo as documentações dos SGBD? Você aproveitou a oportunidade e buscou ajuda dos monitores?

Julgo que consegui desenvolver bastante coisa por conta própria. Foram bastantes pesquisas e leituras de postagens em blogs e fóruns sobre o assunto até conseguir formular uma resposta válida. Em geral eu gostei de ter sido colocado em uma posição de um “pesquisador acadêmico” e ter feito isso tudo sozinho.

Tirei algumas dúvidas com o professor também que me ajudou bastante. Acho que a pior parte foi usar o dbServer no meu computador, que não é um dos melhores, e conseguir entender o DBeaver e restaurar algumas cópias de segurança por causa de alguns problemas que enfrentei, mas no final acredito que desenvolvi corretamente